Domain-Specific Accelerators; Custom RISC-V Extensions

Guided Exercise: XHIST Sparse Histogram Acceleration with QEMU and gem5

Student handout

Exercise goal: Students use a ready Docker image to run a baseline RISC-V workload and an XHIST custom-instruction workload, validate both in QEMU, simulate both in gem5, collect architecture statistics, and reason about whether a custom ISA extension actually improves a non-GeMM workload.

Item	Value
Course	Open-Source Prototype Systems and Applications
Lecture topic	Domain-Specific Accelerators; Custom RISC-V Extensions
Prepared Docker image	amansinhaatnycu/osp:xhist-2026
Workload	Sparse histogram with range filtering and saturating counters
Tools	QEMU user-mode functional emulation; gem5 RISC-V architectural simulation
Expected duration	Approximately 3 hours, including explanation, execution, analysis, and discussion

1. Learning Outcomes

- **Custom ISA motivation:** Identify when repeated computation patterns justify custom instruction support.
- **Functional versus timing tools:** Use QEMU to validate correctness and gem5 to study architectural effects.
- **Instruction encoding:** Understand how a RISC-V custom opcode is emitted by inline assembly and decoded by tools.
- **Workload instrumentation:** Compare dynamic instruction count, cycles, IPC, cache misses, and distribution sensitivity.
- **Accelerator realism:** Explain why fewer instructions do not always imply proportional speedup.
- **Open-source workflow:** Connect source-level changes, simulator extensions, Docker packaging, and reproducible experiments.

What students should know before starting

- **Basic RISC-V execution:** registers, instructions, loads/stores, branches, and integer arithmetic.
- **Basic C compilation:** Makefile usage, static binaries, and command-line execution.
- **Basic Docker usage:** pull, run, bind mount, and command execution inside a container.
- **Basic performance vocabulary:** instruction count, cycles, IPC, cache misses, and bottlenecks.

2. Three-Hour Session Plan

Time	Activity	Student output
09:00-09:20	Instructor explanation: domain-specific accelerators, custom ISA extensions, QEMU vs gem5	Concept notes and initial questions
09:20-09:40	Docker image setup and workload inspection	Container running; source files located
09:40-10:05	QEMU tutorial and functional validation	Baseline and XHIST checksums match
10:05-10:35	gem5 tutorial and first timing runs	stats.txt files generated for both binaries
10:35-11:05	Metric extraction and interpretation	Comparison table: simInsts, cycles, IPC, cache misses
11:05-11:35	Extended tasks: input distributions, cache sweep, xhpack microtest, and CPU model comparison	At least two extended experiments completed
11:35-11:55	Design discussion: next custom instruction or accelerator boundary	Short design proposal
11:55-12:00	Submission checklist and wrap-up	Final report outline ready

3. Workload Overview: Sparse Histogram with XHIST

The workload counts how many input values fall into each histogram bin. Each input element passes through a range check, a bin computation, and a saturating counter update. This is not a GeMM workload; it is a control-heavy and memory-sensitive kernel with irregular updates.

Baseline computation

```
for (uint32_t i = 0; i < N; i++) {
    uint32_t x = input[i];

    if (x >= min_val && x < max_val) {
        uint32_t bin = (x - min_val) >> shift;

        if (hist[bin] < SAT_LIMIT) {
            hist[bin]++;
        }
    }
}
```

XHIST version

```
if (xhrange(x, range_cfg)) {
    uint64_t bin = xhbin(x, bin_cfg);
    hist[bin] = xhsat(hist[bin], SAT_LIMIT);
}
```

Instruction	Operation	What it replaces	Student interpretation
Xhrange	Return 1 if value is in configured range	Two comparisons and a Boolean conjunction	Control-flow reduction
Xhbin	Compute bin index from value and packed config	Subtract and variable shift	Datapath fusion
Xhsat	Increment value unless saturation limit is reached	Compare, branch/select, add	Saturating arithmetic specialization
Xhpack	Pack two 16-bit fields into one register	Shift, mask, and OR	Data-layout helper for compact metadata

Important lesson: The XHIST instructions reduce computation and control overhead, but the histogram update still touches memory. Therefore, the speedup depends on whether execution was compute/control limited or memory limited.

4. Docker Setup and File Locations

Use the prepared image. Students should not rebuild QEMU or gem5 during this exercise unless the instructor explicitly asks them to do so.

```
docker pull amansinhaatnycu/osp:xhist-2026

docker run --rm -it \
  -v "$PWD:/work" \
  amansinhaatnycu/osp:xhist-2026 \
  bash
```

Important paths inside the container

Path	Purpose
/opt/osp-xhist/workloads	Baseline and XHIST C source files, intrinsic header, and Makefile
/opt/osp-xhist/scripts	Run scripts for QEMU, gem5, and stats collection
/opt/osp-xhist/qemu	Patched QEMU source tree and build output
/opt/osp-xhist/gem5	Patched gem5 source tree and build output
/work/results	Student-visible output directory mounted from the host machine

```
cd /opt/osp-xhist/workloads
ls
make clean && make all
```

5. Student Task List: Required and Extended

Task	Required?	What students do	Evidence to submit
Task 1: Setup and compile	Yes	Pull image, run container, compile both binaries	Screenshot or terminal log showing successful make
Task 2: QEMU correctness validation	Yes	Run baseline and XHIST in QEMU	Matching checksums
Task 3: Static instruction inspection	Yes	Use objdump to locate .insn-generated custom instructions	Short note explaining which custom opcodes appear
Task 4: gem5 baseline/XHIST simulation	Yes	Run both binaries under gem5 with the same CPU/cache configuration	Two stats.txt files
Task 5: Metric table	Yes	Compare simInsts, cycles, IPC, and cache misses	One filled comparison table
Task 6: Input distribution experiment	Yes	Create or select different input distributions: uniform, clustered, hot-bin, out-of-range heavy	Explain when XHIST helps more or less
Task 7: Cache sensitivity sweep	Yes	Repeat gem5 runs with at least two L1D cache sizes	Table and one-paragraph interpretation
Task 8: xhpack microtest	Yes	Write a tiny C test that calls xhpack and checks the packed result	Test output and one-line explanation
Task 9: CPU model comparison	Optional challenge	Repeat one pair of runs with MinorCPU or another supported CPU model	Explain why results differ
Task 10: New extension proposal	Optional challenge	Design one additional custom instruction for this or another non-GeMM workload	Encoding, semantics, and expected benefit

6. QEMU Introduction: Functional Emulation for Correctness

QEMU is used here as a fast functional emulator. In user-mode emulation, QEMU can launch a program compiled for one CPU architecture on a host running another architecture. In this exercise, the RISC-V binary runs through qemu-riscv64-xhist on your host/container environment.

Why QEMU first: QEMU quickly answers whether the binary executes and whether the custom instruction semantics are correct. It is not the main source of cycle-level performance claims in this lab.

QEMU role in this exercise

Question	QEMU answer	Not answered by QEMU alone
Does the RISC-V binary run?	Yes, QEMU executes the binary functionally	No cycle-accurate pipeline timing
Do custom instructions decode?	Yes, if decodetree and translator functions are correct	No detailed cache timing interpretation
Do baseline and XHIST outputs match?	Yes, compare checksums	No proof of architectural speedup
Can students debug quickly?	Yes, fast runs and simple terminal output	No detailed microarchitectural bottleneck diagnosis

Run QEMU validation

```
/opt/osp-xhist/scripts/run_qemu_baseline.sh
/opt/osp-xhist/scripts/run_qemu_xhist.sh
```

Expected result: The printed checksum must match between baseline and XHIST. If it does not match, the custom instruction version is functionally wrong even if it looks faster.

QEMU extension pipeline

Stage	File or concept	What happened
Instruction pattern	target/riscv/insn32.decode	Added fixed-bit patterns for xhrange, xhbin, xhsat, and xhpack using custom-0 opcode 0x0b
Decoder generation	QEMU decodetree	QEMU generated decoder code from the instruction patterns
Translation	target/riscv/insn_trans/trans_rvi.c.inc	Added trans_xhrange, trans_xhbin, trans_xhsat, and trans_xhpack
TCG operations	Tiny Code Generator IR	Translator emitted TCG operations for mask, shift, compare, add, select, and OR
Executable	qemu-riscv64-xhist	Built a RISC-V user-mode emulator that understands the XHIST opcodes

Connection to next projects: If students add a new RISC-V instruction in QEMU later, they must define its binary pattern, add the translation function, rebuild QEMU, and validate using a tiny test before running a full workload.

7. QEMU Tutorial Commands

Compile and inspect binaries

```
cd /opt/osp-xhist/workloads
make clean && make all

file build/hist_baseline.riscv
file build/hist_xhist.riscv
```

Inspect disassembly

```
riscv64-linux-gnu-objdump -d build/hist_baseline.riscv > /work/baseline.dump
riscv64-linux-gnu-objdump -d build/hist_xhist.riscv > /work/xhist.dump

grep -n "\.insn" /work/xhist.dump
```

Note: Objdump may not print friendly names such as xhrange or xhbin, because the standard disassembler does not know the new mnemonic. Students should still see machine encodings corresponding to custom opcode space or unfamiliar instructions near the XHIST function calls.

QEMU quick checks

```
qemu-riscv64-xhist --version
/opt/osp-xhist/scripts/run_qemu_baseline.sh
/opt/osp-xhist/scripts/run_qemu_xhist.sh
```

8. gem5 Introduction: Architectural Simulation

gem5 is used here for architectural simulation. Unlike QEMU, gem5 exposes architectural statistics such as simulated instructions, cycles, IPC, and cache behavior. This lets students ask whether the custom instruction changed the system-level performance story, not merely whether the program was correct.

In this lab, students run gem5 in syscall-emulation style with a RISC-V user program. The operating system is not fully booted; gem5 directly models the execution of the target binary and selected system-call behavior. This keeps the exercise focused on ISA extension and workload-level measurement.

gem5 role in this exercise

Question	gem5 contribution
Did XHIST reduce dynamic instruction count?	Compare simInsts between baseline and XHIST
Did fewer instructions reduce cycles?	Compare system.cpu.numCycles
Did efficiency improve?	Compare IPC, but interpret alongside cycles and instruction count
Did memory behavior dominate?	Compare L1D/I-cache misses and cache sweep results
Is speedup input-dependent?	Repeat with different input distributions

gem5 custom instruction extension pipeline

Stage	File or concept	What happened
ISA description	src/arch/riscv/isa/decoder.isa	Added an XHIST decode block in the custom opcode area
Opcode mapping	custom-0 0x0b	gem5 decodes the major opcode after low quadrant bits; the block was placed under 0x02
Instruction semantics	format Rop	Each XHIST instruction writes Rd based on Rs1 and Rs2 values
Generated decoder	build/RISCV/arch/riscv/generated/decoder.cc	gem5 generated C++ decoder code from decoder.isa
Simulation binary	build/RISCV/gem5.opt	Rebuilt gem5 with XHIST-aware RISC-V decode support

Connection to next projects: For a future project, students can design a custom instruction, add QEMU functional semantics, add gem5 architectural semantics, compile a workload using inline assembly, then compare correctness and timing behavior.

9. gem5 Tutorial Commands

Run baseline and XHIST

```
mkdir -p /work/results

/opt/osp-xhist/scripts/run_gem5_baseline.sh
/opt/osp-xhist/scripts/run_gem5_xhist.sh
```

Check output files

```
ls -lh /work/results/gem5_baseline/stats.txt
ls -lh /work/results/gem5_xhist/stats.txt
```

Extract key statistics

```
grep -E "simInsts|numCycles|ipc|overallMisses" \
  /work/results/gem5_baseline/stats.txt

grep -E "simInsts|numCycles|ipc|overallMisses" \
  /work/results/gem5_xhist/stats.txt
```

Use the provided collection script

```
python3 /opt/osp-xhist/scripts/collect_stats.py \
  /work/results/gem5_baseline/stats.txt \
  /work/results/gem5_xhist/stats.txt
```

Student metric table

Metric	Baseline	XHIST	Interpretation
simInsts			Did custom instructions reduce dynamic instruction count?
system.cpu.numCycles			Did total simulated cycles decrease?
system.cpu.ipc			Did useful instruction throughput change?
L1D misses			Was the workload still memory sensitive?
L1I misses			Did code footprint or control flow change?

10. Required Task 1: Build and Functional Validation

Task instructions

- **Compile both binaries:** Build hist_baseline.riscv and hist_xhist.riscv using make.
- **Run both binaries in QEMU:** Use the prepared scripts to validate functional correctness.
- **Check output:** The checksum values must match exactly.
- **Record evidence:** Copy terminal output into your report.

```
cd /opt/osp-xhist/workloads
make clean && make all

/opt/osp-xhist/scripts/run_qemu_baseline.sh
/opt/osp-xhist/scripts/run_qemu_xhist.sh
```

Questions to answer

- What does the checksum prove, and what does it not prove?
- Why is QEMU the right first tool before gem5?
- Why should we not claim speedup from QEMU runtime alone?

11. Required Task 2: Static Instruction Inspection

Students inspect whether the XHIST binary actually contains instructions generated from inline assembly. This task connects C source code to machine encoding.

```
cd /opt/osp-xhist/workloads

riscv64-linux-gnu-objdump -d build/hist_baseline.riscv > /work/baseline.dump
riscv64-linux-gnu-objdump -d build/hist_xhist.riscv > /work/xhist.dump

# Look near main or near the XHIST helper calls.
grep -n "<main>" /work/xhist.dump
less /work/xhist.dump
```

Questions to answer

- Where do the custom instructions appear relative to the histogram loop?
- Why might objdump not know the friendly XHIST mnemonic names?
- How does inline assembly let us avoid modifying GCC for this lab?

12. Required Task 3: gem5 Baseline versus XHIST

```
mkdir -p /work/results

/opt/osp-xhist/scripts/run_gem5_baseline.sh
/opt/osp-xhist/scripts/run_gem5_xhist.sh

python3 /opt/osp-xhist/scripts/collect_stats.py \
  /work/results/gem5_baseline/stats.txt \
  /work/results/gem5_xhist/stats.txt
```

Interpretation prompts

- **Instruction reduction:** Did XHIST lower simInsts? Explain which operations were fused.
- **Cycle reduction:** Did cycles decrease by the same percentage as instruction count? Explain why or why not.
- **IPC shift:** Did IPC improve, decrease, or remain similar? Relate this to instruction count and memory behavior.
- **Memory bottleneck:** Did L1D misses remain important after the custom instructions?

13. Required Task 4: Input Distribution Experiment

The same custom instruction can help differently depending on input distribution. Students should test whether the workload is dominated by range checks, bin computation, or histogram memory updates.

Minimum required distributions

Distribution	Description	Expected behavior to examine
Uniform in range	Most values valid and spread across bins	Many histogram updates; memory behavior may dominate
Hot-bin clustered	Many values map to a small number of bins	Fewer unique bins; possible better locality but saturation effects matter
Out-of-range heavy	Many values fail range check	Range filtering becomes more important; XHIST may reduce branch/compare overhead
Adversarial stride	Values produce scattered bins	Irregular access stresses cache behavior

Suggested modification point

Modify the init_input() function in both baseline and XHIST versions. Keep N, NBINS, SAT_LIMIT, min_val, max_val, and shift consistent unless your report clearly states the change.

```
static void init_input(void)
{
    for (uint32_t i = 0; i < N; i++) {
        input[i] = ...;   // Replace this expression for each distribution.
    }
}
```

Report questions

- Which input distribution gave the largest XHIST benefit?
- Which distribution made XHIST least useful?
- Did locality or branch/control reduction dominate the result?

14. Required Task 5: Cache Sensitivity Sweep

A custom instruction may reduce arithmetic and control instructions, but cache behavior can still dominate. Students should repeat gem5 simulations with different L1D cache sizes.

Edit run scripts or run manually

```
# Example manual run with smaller L1D cache
mkdir -p /work/results/gem5_xhist_l1d_8k

gem5-riscv-xhist \
  --outdir=/work/results/gem5_xhist_l1d_8k \
  /opt/osp-xhist/gem5/configs/deprecated/example/se.py \
  --cmd=/opt/osp-xhist/workloads/build/hist_xhist.riscv \
  --cpu-type=TimingSimpleCPU \
  --caches \
  --l1d_size=8kB \
  --l1i_size=32kB \
  --cacheline_size=64
```

Configuration	Baseline cycles	XHIST cycles	Baseline L1D misses	XHIST L1D misses	Conclusion
L1D 8kB					
L1D 32kB					
L1D 64kB					

15. Required Task 6: xhpack Microtest

The provided workload mainly uses xhrange, xhbin, and xhsat. This task forces students to directly test xhpack and understand how small data-layout helpers can be custom instructions.

Create a tiny test file

```
cd /opt/osp-xhist/workloads
cat > test_xhpack.c <<'EOF'
#include <stdint.h>
#include <stdio.h>
#include "xhist_intrin.h"

int main(void)
{
    uint64_t a = 0x1234;
    uint64_t b = 0xabcd;
    uint64_t r = xhpack(a, b);
    printf("xhpack result: 0x%lx\n", r);
    return 0;
}
EOF

riscv64-linux-gnu-gcc -O2 -static -march=rv64gc -mabi=lp64d \
  -o build/test_xhpack.riscv test_xhpack.c

qemu-riscv64-xhist build/test_xhpack.riscv
```

Expected reasoning

For a = 0x1234 and b = 0xabcd, xhpack returns ((a & 0xffff) << 16) | (b & 0xffff). Students should compute the expected value manually and compare it with QEMU output.

16. Optional Challenge: CPU Model Comparison

Repeat one baseline/XHIST pair with another gem5 CPU model supported by the image. Use caches for more detailed CPU models.

```
mkdir -p /work/results/gem5_xhist_minor

gem5-riscv-xhist \
  --outdir=/work/results/gem5_xhist_minor \
  /opt/osp-xhist/gem5/configs/deprecated/example/se.py \
  --cmd=/opt/osp-xhist/workloads/build/hist_xhist.riscv \
  --cpu-type=MinorCPU \
  --caches \
  --l1d_size=32kB \
  --l1i_size=32kB \
  --cacheline_size=64
```

- **Compare with TimingSimpleCPU:** Does the same instruction reduction produce the same performance trend?
- **Explain model sensitivity:** A CPU model changes timing assumptions, pipeline behavior, and stall visibility.

17. Optional Challenge: Design a New Custom Instruction

Students propose one new custom instruction for XHIST or a different non-GeMM workload. They do not have to implement it during the 3-hour lab, but the design must be concrete enough to implement later.

Field	Required content
Instruction name	Example: xhclamp, xhidxsat, xh2bin, xhcmpmask, xhreduce4
Operands	Define rd, rs1, rs2 meaning
Encoding	Use custom-0 or another custom opcode space; specify funct3/funct7
Semantics	Precise C-like operation
Why it helps	Which repeated baseline sequence it replaces
Risk	What it hides or makes unrealistic: memory side effects, latency, exceptions, ordering, or compiler difficulty

Example proposal format

```
Instruction: xhclamp rd, rs1, rs2
Meaning: rd = min(max(rs1, lo), hi), where rs2 packs lo and hi.
Encoding: opcode custom-0, funct3 = 0x4, funct7 = 0x01.
Expected benefit: replaces two comparisons and two selects in clipping-heavy kernels.
Concern: may not help if the workload is dominated by memory traffic.
```

18. Optional Challenge: Workload Transfer

Choose another non-GeMM workload and identify whether XHIST-style specialization would be useful.

Candidate workload	Possible custom instruction idea	Why it is domain-specific
CRC/checksum	Fused byte update or partial polynomial step	Repeated bitwise recurrence
Run-length encoding	Compare-run-update instruction	Branch-heavy compression loop
Sparse gather/filter	Predicate and compact index helper	Irregular data selection
Image threshold histogram	Range, clamp, and bin helper	Repeated pixel binning
Packet classification	Mask-compare instruction	Repeated header field checks

19. Report Template

Students should submit a concise report. The report should be evidence-driven: outputs and stats first, claims second.

Required report sections

- **1. Setup summary:** Docker image tag, date, and whether commands completed successfully.
- **2. Workload explanation:** Baseline histogram loop and XHIST-transformed loop.
- **3. QEMU validation:** Baseline checksum, XHIST checksum, and correctness conclusion.
- **4. gem5 metrics:** Table comparing simInsts, cycles, IPC, and cache statistics.
- **5. Distribution experiment:** At least two input distributions and their effect on performance.
- **6. Cache sweep:** At least two L1D sizes and interpretation.
- **7. xhpack microtest:** Expected and observed packed result.
- **8. Design reflection:** One additional custom instruction or accelerator idea.
- **9. Final conclusion:** Did XHIST accelerate the workload? Under what conditions?

Minimum tables to include

Table	Required columns
QEMU correctness	binary, command, checksum, pass/fail
gem5 baseline vs XHIST	metric, baseline, XHIST, percent change, interpretation
Input distributions	distribution, baseline cycles, XHIST cycles, speedup, explanation
Cache sweep	L1D size, cycles, L1D misses, conclusion

20. Troubleshooting Guide

Problem	Likely cause	Fix
qemu-riscv64-xhist not found	Symlink missing or wrong image	Use the xhist-2026 image; check /usr/local/bin/qemu-riscv64-xhist
Illegal instruction in QEMU	Wrong QEMU binary or custom instruction not decoded	Use qemu-riscv64-xhist, not system qemu-riscv64
Checksums differ	XHIST code or semantics incorrect	Recompile; inspect xhist_intrin.h; confirm same input generation
gem5 script path fails	Using old configs/example/se.py path	Use /opt/osp-xhist/gem5/configs/deprecated/example/se.py
stats.txt missing	gem5 run failed before completion	Read /work/results/.../simout and simerr
O3CPU error	O3 model may require caches or compatible options	Use --caches or choose TimingSimpleCPU/MinorCPU
No speedup	Workload may be memory-bound	Use cache sweep and distribution experiment to explain bottleneck

Useful commands for debugging

```
which qemu-riscv64-xhist
which gem5-riscv-xhist

ls -lh /opt/osp-xhist/workloads/build
ls -lh /work/results/gem5_baseline
cat /work/results/gem5_baseline/simout | tail
cat /work/results/gem5_baseline/simerr | tail
```

22. Instructor Notes: What Students Should Learn

- **Custom instructions are contracts:** QEMU, gem5, compiler/assembly, and workload code must agree on encoding and semantics.
- **Functional correctness comes before performance:** QEMU checksum mismatch invalidates the experiment.
- **Simulation metrics require interpretation:** Lower simInsts is useful, but memory behavior can cap speedup.
- **Non-GeMM workloads matter:** Histograms, filters, packet processing, compression, and checksums often expose control and irregular-memory bottlenecks.
- **Extensions are not magic:** A useful custom instruction should remove a common critical sequence without hiding unrealistic memory behavior.

23. References

QEMU Documentation, User Mode Emulation: https://qemu-project.gitlab.io/qemu/user/index.html

QEMU Documentation, Decodetree Specification: https://www.qemu.org/docs/master/devel/decodetree.html

QEMU Documentation, Translator Internals / TCG: https://www.qemu.org/docs/master/devel/tcg.html

gem5 Documentation, ISA Parser: https://www.gem5.org/documentation/general_docs/architecture_support/isa_parser/

gem5 Project Website: https://www.gem5.org/